

Informe Ejercicio 2

Algoritmo Determinante de una Matriz $N \times N$

El presente informe describe el desarrollo de un algoritmo que permite generar una matriz cuadrada de tamaño $N \times N$ y calcular su determinante. La matriz se llena automáticamente con números aleatorios entre 1 y 20 mediante el método `fill_matrix()`. El programa fue desarrollado aplicando buenas prácticas de programación basadas en el estándar PEP8, utilizando funciones, clases, nombres descriptivos y pruebas unitarias para validar el correcto funcionamiento del algoritmo.

1. Descripción del Problema

Se requiere desarrollar un algoritmo que permita:

- i. Crear una matriz cuadrada de tamaño $N \times N$.
- ii. Llenar automáticamente la matriz con números aleatorios entre 1 y 20.
- iii. Calcular el determinante de la matriz.
- iv. Mostrar la matriz generada y el resultado del determinante.

2. Desarrollo de la Solución

El problema se resolvió mediante el uso de una clase llamada `Matrix`, la cual permite organizar el código y agrupar las funcionalidades relacionadas con la matriz.

La solución incluye los siguientes métodos:

- i. `fill_matrix()`: llena la matriz con números aleatorios entre 1 y 20.
- ii. `calculate_determinant()`: calcula el determinante de la matriz.
- iii. `get_minor()`: obtiene matrices menores necesarias para el cálculo del determinante.
- iv. `show_matrix()`: muestra la matriz en pantalla.
- v. `main()`: coordina la ejecución principal del programa.

El algoritmo calcula el determinante utilizando operaciones matemáticas sobre matrices cuadradas. Para matrices pequeñas como 2×2 , aplica directamente la fórmula matemática correspondiente. Para matrices más grandes, divide el problema en matrices más pequeñas hasta obtener el resultado final.

3. Código Implementado

```
import random
```

```
class Matrix:
```

```
    """
```

```
    Representa una matriz cuadrada de tamaño  $N \times N$ .
```

```
    """
```

```
    def __init__(self, size):
```

```
        self.size = size
```

```

self.data = [ ]

def fill_matrix(self):
    """
    Llena la matriz con números aleatorios entre 1 y 20.
    """
    self.data = [ ]

    for _ in range(self.size):
        row = [ ]

        for _ in range(self.size):
            number = random.randint(1, 20)
            row.append(number)

        self.data.append(row)

def calculate_determinant(self, matrix=None):
    """
    Calcula el determinante de una matriz.
    """
    if matrix is None:
        matrix = self.data

    size = len(matrix)

    if size == 1:
        return matrix[0][0]

    if size == 2:
        return (
            matrix[0][0] * matrix[1][1]
            - matrix[0][1] * matrix[1][0]
        )

    determinant = 0

    for column in range(size):
        minor = self.get_minor(matrix, 0, column)
        sign = (-1) ** column

        determinant += (
            sign
            * matrix[0][column]
            * self.calculate_determinant(minor)
        )

```

```
return determinant
```

4. Prueba de Escritorio

Se utilizó la siguiente matriz de prueba:

$$\begin{vmatrix} 3 & 2 \\ 5 & 4 \end{vmatrix}$$

Para una matriz de tamaño 2 x 2, el determinante se calcula con la siguiente fórmula:

$$(a * d) - (b * c)$$

Reemplazando los valores:

$$\begin{aligned} &(3 * 4) - (2 * 5) \\ &12 - 10 \\ &2 \end{aligned}$$

Resultado esperado:

Determinante = 2

5. Pruebas Unitarias

Con el fin de validar el correcto funcionamiento del algoritmo, se implementaron pruebas unitarias utilizando la librería unittest de Python.

Las pruebas permiten verificar que el algoritmo funcione correctamente en diferentes escenarios:

- Matrices de 1 x 1.
- Matrices de 2 x 2.
- Matrices de 3 x 3.
- Validación del tamaño de la matriz generada.
- Verificación de que los números aleatorios estén entre 1 y 20.

```
import unittest
```

```
from matrix import Matrix
```

```
class TestMatrix(unittest.TestCase):
```

```
    def test_determinant_1x1(self):  
        matrix = Matrix(1)  
        matrix.data = [[5]]
```

```

        result = matrix.calculate_determinant()

        self.assertEqual(result, 5)

    def test_determinant_2x2(self):
        matrix = Matrix(2)
        matrix.data = [
            [3, 2],
            [5, 4]
        ]

        result = matrix.calculate_determinant()

        self.assertEqual(result, 2)

    def test_fill_matrix_values_range(self):
        matrix = Matrix(3)
        matrix.fill_matrix()

        for row in matrix.data:
            for value in row:
                self.assertGreaterEqual(value, 1)
                self.assertLessEqual(value, 20)

if __name__ == "__main__":
    unittest.main()

```

Al ejecutar las pruebas unitarias, todas las validaciones fueron exitosas, confirmando que el algoritmo calcula correctamente el determinante y genera matrices válidas.

6. Resultado Final

Después de ejecutar el programa, se genera automáticamente una matriz cuadrada con números aleatorios entre 1 y 20 y posteriormente se calcula su determinante.

Ejemplo:

Matriz generada:

```

[3, 2]
[5, 4]

```

Determinante = 2

7. Conclusión

El desarrollo de este ejercicio permitió entender mejor cómo se organiza una matriz y cómo obtener un resultado a partir de sus datos. Además, ayudó a practicar el uso de Python de una forma más clara y ordenada.